Rutwiya SafeCheck: A Safety Verifier for Open Multi Agent Systems

Alphy George, Ajay Kumar, Pranav Prashant, Shabana AT, S Sheerazuddin, Vipin S Gautam

***Abstract*—In [5] we proposed a regular version of open multi agent systems (OMAS) [10] where agents can leave only from a particular *leave* state. We also showed (by a reduction to multi-counter systems) that reachability and, therefore, safety checking is decidable in regular OMAS. In this paper we report an encoding scheme for regular OMAS into Petri nets [3] and an implementation of a safety verifier for the same. Given a regular OMAS and a safety criterion as input, our tool, named Rutwiya SafeCheck, using *its-reach* of the ITS-tools [4] suite as a backend, checks whether any unsafe state of agents in the given input is reachable or not. We also observe that the encoding scheme, described herein, can be easily extended to general OMAS [10].**

***Index Terms*—Formal Verification, Safety Verification, Multi-Agent Systems, Petri nets, Counter Abstraction**

I. Introduction

MULTI-agent systems (MAS) [21] have emerged as a powerful model for designing distributed, autonomous systems where multiple agents interact and cooperate to achieve shared goals. Such systems are found in a wide range of domains, including robotics, intelligent transport, distributed sensor networks, and swarm-based surveillance. A major verification challenge in MAS arises when the number of participating agents is not fixed at design time. This leads to the notion of open multi-agent systems (OMAS)—systems in which agents may dynamically enter or exit during runtime [10]. While OMAS more accurately capture real-world agent deployments, they significantly increase the complexity of formal verification.

Recent research has shown that general verification problems, such as model checking, are undecidable for OMAS [10]. This motivates the need to identify subclasses of OMAS where automated verification is feasible. In earlier work [5], we proposed one such subclass, called *regular* open multi-agent systems, or regular OMAS, where agent departures are restricted to occur only from designated *leave* states. This constraint enables a counter abstraction of the system's behavior, allowing its semantics to be captured by multi-counter systems—a model with decidable reachability properties. Our prior work demonstrated that for regular OMAS, safety verification becomes decidable through this abstraction.

Despite these theoretical advances, practical tools for verifying safety properties in such systems remain scarce. The absence of such tools limits the deployment of formally verified MAS in real-world safety-critical scenarios, such as industrial inspection robotics or disaster-response swarms.

This paper takes a step toward bridging that gap. We present a novel encoding of regular OMAS into Petri nets [3], a well-established formalism with rich tool support. Petri nets are particularly suited to modeling concurrency, dynamic resource allocation, and synchronization—all core features of MAS. Leveraging this encoding, we introduce *Rutwiya SafeCheck*, a tool that performs automated safety verification for regular OMAS by translating the input model into a Petri net and invoking the *its-reach* engine from the ITS-tools suite as a backend solver.

Given a regular OMAS and a user-specified safety property, Rutwiya SafeCheck determines whether any unsafe agent state is reachable, thereby providing formal safety guarantees—or identifying potential violations. The tool supports a range of practical scenarios where agents join or exit at runtime, and where safety-critical conditions must be enforced.

We can summarize the contributions of this paper as follows (1) a formal encoding scheme that translates regular OMAS into Petri nets, preserving the system's operational semantics, (2) a rigorous proof of correctness of the encoding scheme, (3) an automated verification tool—*Rutwiya SafeCheck*—that integrates with existing Petri net reachability solvers to check safety properties, also (4) an evaluation of Rutwiya SafeCheck on small-scale regular OMAS scenarios, demonstrating its correctness and usability.

(Parts of the contents of this section were paraphrased by [23]).

II. Open MAS

Open multi-agent systems (open MAS) are characterized by the dynamic participation of agents, where new agents may join or existing agents may leave the system during execution. In this section, we present a formal framework for modeling such systems, based on the Open Interpreted System (OIS) model introduced by [10] and originally grounded in interpreted systems by [9]. These models offer a foundation for reasoning about temporal and epistemic aspects of multi-agent and distributed systems.

A. Syntax and Semantics of Open MAS

The agents in an open MAS are defined by an agent template as follows.

Definition 1 (Agent)**.** *An agent template is a tuple* $AT = \langle L, \iota, Act, P, tr \rangle$ *where*

- $L \neq \emptyset$ *is a finite set of local states,*
- $\iota \in L$ *is the initial local state,*
- Act *is a non-empty set of actions, including a special action* τ *indicating inaction,*
- $P : L \to \mathcal{P}(Act)$ *defines the enabled actions at each state, with* $\tau \in P(l)$ *for all* $l \in L$*,*
- $tr : L \times Act \times \mathcal{P}(Act) \times Act_E \to L$ *is a transition function computing the next local state based on the agent's action, actions of other agents, and the environment's action.*

This paper is produced by the verification group at National Institute of Technology, Calicut, India

The environment is specified in a similar formal manner.

Definition 2 (Environment)**.** *The environment is defined as* $E = \langle L_E, \iota_E, Act_E, P_E, tr_E \rangle$ *where*

- L_E *is the set of environment states, disjoint from agent states,*
- ι_E *is the initial environment state,*
- Act_E *is the set of environment actions,*
- $P_E : L_E \to \mathcal{P}(Act_E)$ *specifies enabled environment actions,*
- $tr_E : L_E \times Act_E \times \mathcal{P}(Act) \to L_E$ *determines the next state of the environment.*

We combine the agent and environment specifications to define an Open Interpreted System.

Definition 3 (OIS)**.** *An Open Interpreted System (OIS) is a structure* $\mathcal{O} = \langle AT, E, V \rangle$*, with agent, environment and a valuation function* $V : L \to \mathcal{P}(AP)$*.*

At any moment, the system may consist of the environment and a varying number of active agents. Each agent is associated with a unique identifier upon joining. The system's global state aggregates the current states and identifiers of all agents, along with the environment state.

We use $[n]$ to denote the set $\{1, 2, \ldots, n\}$. Let G_n denote the set of global states when n agents are present. Each global state is a tuple in $(L \times \mathbb{Z}^+) \times \cdots \times (L \times \mathbb{Z}^+) \times L_E$. For a state $g \in G_n$, $g.j$ refers to the j^{th} agent's state and $id(g.j)$ its identifier. The environment's state is denoted $g.E$. The overall state space is given by $G = \bigcup_{n \in \mathbb{N}} G_n$.

The set of joint actions are defined over n agents and the environment as $ACT_n = Act^n \times Act_E$. Given a joint action $a \in ACT_n$, the projection onto agent actions is denoted $\dot{a} = \{a.j \mid 1 \leq j \leq n\}$, and $\dot{a}_{-i}$ denotes the set of actions by all agents except agent i.

We now formalize the system's dynamics through global transition relations.

Definition 4 (Global transition relation of size n)**.** *Let* $R_n \subseteq G_n \times G_n$*. Then,* $(g, g') \in R_n$ *if there exists* $a \in ACT_n$ *such that*

- $a.E \in P_E(g.E)$ *and* $a.j \in P(g.j)$ *for all* $j \in [n]$*,*
- $g'.E = tr_E(g.E, a.E, \dot{a})$*,*
- *For all* $j \in [n]$*,* $id(g'.j) = id(g.j)$ *and* $g'.j = tr(g.j, a.j, \dot{a}_{-j}, a.E)$*.*

This ensures consistency with the protocol and transition rules of both agents and the environment.

Definition 5 (Global transition relation)**.** *The overall global transition relation* $R \subseteq G \times G$ *includes three cases*

> **Joint action:** $(g, g') \in R_n$ *for some* $n \in \mathbb{N}$*,*
> **Agent joining:** $g \in G_n$*,* $g' \in G_{n+1}$ *where the environment and existing agent states remain unchanged, and a new agent is added to the initial state* ι *with a unused identity.*
> **Agent leaving:** $g \in G_n$*,* $g' \in G_{n-1}$ *obtained by removing one agent's component from* g*.*

B. regular OMAS

In regular OMAS, agents are only allowed to exit the system from a designated state called the *leave state*. The agent template is modified to include this: $AT = \langle L, \iota, Act, P, tr, leave \rangle$ where $leave \in L$ and $P(leave) = \emptyset$ (i.e., no further actions are enabled).

The agent leaving rule is revised as

- **Agent leaving:** $g \in G_n$, $g' \in G_{n-1}$, and there exists agent i such that $g.i = leave$ and $g' = g_{-\{i\}}$.

(Portions of this section are adapted from our earlier work [4] for clarity and completeness.)

III. Safety Checking in regular OMAS

We give a decision method for safety checking in regular OMAS by encoding them into Petri nets [3] and invoking the corresponding reachability algorithm [2] [1]. We first revisit the notion of Petri nets. A Petri net N is defined formally as a 5-tuple $N = (P, T, F, W, M_0)$ where

- P is a finite set of *places*,
- T is a finite set of *transitions*,
- F is a *flow relation* over places and transitions, $F \subseteq (P \times T) \cup (T \times P)$,
- W is the *weight function*, $W : F \longrightarrow \mathbb{N}$ and
- $M_0 : P \longrightarrow \mathbb{N}$ is the *initial marking*.

A *firing sequence* of a Petri net with initial marking M_0 is a sequence of transitions

$$\sigma = \langle t_1 \cdots t_n \rangle$$

such that there exists a run ϱ:

$$\varrho = M_0 \xrightarrow{t_1} M_1 \xrightarrow{t_2} \cdots \xrightarrow{t_{n-1}} M_{n-1} \xrightarrow{t_n} M_n.$$

The set of firing sequences is denoted as $\mathcal{L}(N)$. A marking M is reachable from M_0 if there exists a sequence of firing that transforms M_0 to M. The set of reachable markings for N is denoted as $R(N)$. We can define the set of all runs of N as follows:

$$Runs(N) = \{\varrho = M_0 \xrightarrow{t_1} M_1 \cdots \xrightarrow{t_n} M | M \in R(N)\}.$$

The reachability graph of N is denoted by $\mathcal{G}(N)$ and it consists of nodes as reachable markings and edges as transitions. Note that the runs in $Runs(N)$ are paths in $\mathcal{G}(N)$ starting from M_0. Also, we may define the set of all possible markings for N as follows: $\mathbb{M}(N) = \{M \mid M : P \longrightarrow \mathbb{N}\}$

A. Encoding of regular OMAS into Petri nets

Given a regular OMAS $\mathcal{O} = \langle AT, E, V \rangle$, we define an encoding of $\mathcal{O}$ as an *interpreted* Petri net $N_{\mathcal{O}} = (P_{\mathcal{O}}, T_{\mathcal{O}}, F_{\mathcal{O}}, W_{\mathcal{O}}, M^0_{\mathcal{O}}, \mathcal{V}_{\mathcal{O}})$ that captures the behaviour of $\mathcal{O}$. We proceed to describe the various components of $N_{\mathcal{O}}$.

- $P_{\mathcal{O}} = L \cup L_E \cup \{ok\}$; For every location in agent template and environment, there is a corresponding place in $N_{\mathcal{O}}$. We have added an extra place ok to atomize joint actions from $\mathcal{O}$ in the net $N_{\mathcal{O}}$.
- $M^0_{\mathcal{O}} : P_{\mathcal{O}} \longrightarrow \mathbb{N}$ such that $M^0_{\mathcal{O}}(\iota_E) = 1$ and $M^0_{\mathcal{O}}(ok) = 1$ and for all $l \in (P_{\mathcal{O}} - \{\iota_E, ok\})$, $M^0_{\mathcal{O}}(l) = 0$; Initially

there is a token in the place corresponding to ι_E and also ok, whereas all other places are vacant.

- We initialize $T_{\mathcal{O}}, F_{\mathcal{O}}, W_{\mathcal{O}}$ as follows:
  - $T_{\mathcal{O}} = \{t_{aj}, t_{al}, stop\} \cup \{\tau_l \mid l \in L\}$; t_{aj} is the transition in the net $N_{\mathcal{O}}$ corresponding to agent joining action in $\mathcal{O}$ & t_{al} is the transition corresponding to agent leaving action. The special transition $stop$ is used to signal the end of a joint action. For any $l \in L$, τ_l corresponds to the *silent* action (inaction) from l.
  - $F_{\mathcal{O}} = \{(t_{aj}, \iota), (leave, t_{al}), (stop, ok)\} \cup \{(l, \tau_l), (\tau_l, l) \mid l \in L\}$; t_{aj} is a source transition that adds tokens to the place ι whereas t_{al} is a sink transition that removes tokens from the place $leave$. When $stop$ transition is executed a token moves to ok thereby indicating the possibility of firing another joint action.
  - $W_{\mathcal{O}} = \{(t_{aj}, \iota) \mapsto 1, (leave, t_{al}) \mapsto 1, (stop, ok) \mapsto 1\} \cup \{(l, \tau_l) \mapsto 1, (\tau_l, l) \mapsto 1 \mid l \in L\}$; Only one agent can enter or leave the system hence the weights are 1. Also, only one joint action can happen at any point of time.
- Now for every environment transition tuple $\mathtt{tr} = (l_e, b, A, l'_e)$ in tr_E we augment $T_{\mathcal{O}}, F_{\mathcal{O}}, W_{\mathcal{O}}$ as follows. Corresponding to $\mathtt{tr}$, we generate all possible global joint transitions in $\mathcal{O}$ (denoted by $\mathcal{TR}_{\mathtt{tr}}$) and add them to $T_{\mathcal{O}}$. Further, using the information in $\mathcal{TR}_{\mathtt{tr}}$, we compute the edge relations in $F_{\mathcal{O}}$ and the weights in $W_{\mathcal{O}}$.
  - Let $A = \{a_1, a_2, \ldots, a_k\}$ be the set of all agent actions executed in a joint action corresponding to the aforementioned environment transition, $\mathtt{tr}$. $\mathcal{TR}_{\mathtt{tr}}$ contains tuples of the form $\langle (l_1, \mathfrak{a}_1, A_1, b, l'_1), \ldots, (l_{\mathfrak{n}}, \mathfrak{a}_{\mathfrak{n}}, A_{\mathfrak{n}}, b, l'_{\mathfrak{n}}), (l_e, b, A, l'_e)\rangle$ which must satisfy the following property:
    - $\{\mathfrak{a}_1, \ldots, \mathfrak{a}_{\mathfrak{n}}\} = A$ and $k \leq \mathfrak{n} \leq 2 * k$. Also, for any $a \in A$, a occurs at most twice in the sequence $\langle \mathfrak{a}_1, \ldots, \mathfrak{a}_{\mathfrak{n}}\rangle$
    - For all $1 \leq i \leq \mathfrak{n}$, $l'_i = tr(l_i, \mathfrak{a}_i, A_i, b)$ and $A_i = \{\mathfrak{a}_1, \ldots, \mathfrak{a}_{i-1}, \mathfrak{a}_{i-1}, \ldots, \mathfrak{a}_{\mathfrak{n}}\}$.
  - We segregate $\mathcal{TR}_{\mathtt{tr}}$ into two (disjoint) parts: $\mathcal{TR}^{=1}_{\mathtt{tr}}$ and $\mathcal{TR}^{\leq 2}_{\mathtt{tr}}$. $\mathcal{TR}^{=1}_{\mathtt{tr}}$ contains tuples $\langle (l_1, \mathfrak{a}_1, A_1, b, l'_1), \ldots, (l_{\mathfrak{n}}, \mathfrak{a}_{\mathfrak{n}}, A_{\mathfrak{n}}, b, l'_{\mathfrak{n}}), (l_e, b, A, l'_e)\rangle$ from $\mathcal{TR}_{\mathtt{tr}}$ such that for any $a \in A$, a occurs exactly once in the sequence $\langle \mathfrak{a}_1, \ldots, \mathfrak{a}_{\mathfrak{n}}\rangle$ and $\mathcal{TR}^{\leq 2}_{\mathtt{tr}} = \mathcal{TR}_{\mathtt{tr}} \setminus \mathcal{TR}^{=1}_{\mathtt{tr}}$.
  - First, we define $T_{\mathtt{tr}} = \{t_{\mathfrak{tt}} \mid \mathfrak{tt} \in \mathcal{TR}_{\mathtt{tr}}\}$ as the min-set of transitions in $N_{\mathcal{O}}$ contributed by $\mathtt{tr}$ and, therefore, added to $T_{\mathcal{O}}$. We add some more transitions in $T_{\mathcal{O}}$ as we go on, depending on the plus-gadgets that are plugged in. For any $\mathfrak{tt} = \langle (l_1, \mathfrak{a}_1, A_1, b, l'_1), \ldots, (l_{\mathfrak{n}}, \mathfrak{a}_{\mathfrak{n}}, A_{\mathfrak{n}}, b, l'_{\mathfrak{n}}), (l_e, b, A, l'_e)\rangle$ in $\mathcal{TR}_{\mathtt{tr}}$, if $\mathfrak{tt} \in \mathcal{TR}^{=1}_{\mathtt{tr}}$, we construct a transition (min-gadget + stop) in the Petri net $N_{\mathcal{O}}$ otherwise we construct a transition (min-gadget + plus-gadgets + stop) in the Petri net $N_{\mathcal{O}}$ as follows.
    - Let *pre-transition set* $\mathfrak{pre}(\mathfrak{tt})$ and *post-transition set* $\mathfrak{post}(\mathfrak{tt})$ be defined as:

$$\mathfrak{pre}(\mathfrak{tt}) = \{l_1, l_2, \ldots, l_{\mathfrak{n}}, l_e\}$$

$$\mathfrak{post}(\mathfrak{tt}) = \{l'_1, l'_2, \ldots, l'_{\mathfrak{n}}, l'_e\}$$

      where $\mathfrak{post}(\mathfrak{tt})$ give the set of all *pre-transition places* and *post-transition places* for the corresponding global joint action $\mathfrak{tt}$.
    - Now, for any $l \in \mathfrak{pre}(\mathfrak{tt})$ and $l' \in \mathfrak{post}(\mathfrak{tt})$, let $n^{\mathfrak{tt}}_l$ and $n^{\mathfrak{tt}}_{l'}$ be the number of times l and l' occurs in the tuples $\langle l_1, l_2, \ldots, l_{\mathfrak{n}}\rangle$ and $\langle l'_1, l'_2, \ldots, l'_{\mathfrak{n}}\rangle$, respectively.
    - We add a set $\mathfrak{f}^{\mathfrak{tt}}_{\mathtt{tr}}$ of (flow) edges to $F_{\mathcal{O}}$ as follows: $\mathfrak{f}^{\mathfrak{tt}}_{\mathtt{tr}} = \{(l, t_{\mathfrak{tt}}) \mid l \in \mathfrak{pre}(\mathfrak{tt})\} \cup \{(t_{\mathfrak{tt}}, l') \mid l' \in \mathfrak{post}(\mathfrak{tt})\} \cup \{(ok, t_{\mathfrak{tt}})\}$.
    - We add the following set to weight function $W_{\mathcal{O}}$: $\mathfrak{w}^{\mathfrak{tt}}_{\mathtt{tr}} = \{(l, t_{\mathfrak{tt}}) \mapsto n^{\mathfrak{tt}}_l \mid l \in \mathfrak{pre}(\mathfrak{tt})\} \cup \{(t_{\mathfrak{tt}}, l') \mapsto n^{\mathfrak{tt}}_{l'} \mid l' \in \mathfrak{post}(\mathfrak{tt})\} \cup \{(ok, t_{\mathfrak{tt}}) \mapsto 1\}$.
    - At this point we have computed the min-gadget for the transition $t_{\mathfrak{tt}}$ and depending on the type of $\mathfrak{tt}$ we either need to plugin the stop transition or plus-gadgets for each $a \in A$ and the stop transition. In either situations, we need to add an extra place $p_{\mathfrak{tt}}$ to $P_{\mathcal{O}}$ and add the following set of tuples to $F_{\mathcal{O}}$ each having weight 1: $\{(t_{\mathfrak{tt}}, p_{\mathfrak{tt}}), (p_{\mathfrak{tt}}, stop)\}$. Also, we need to attach the τ transitions at this point by adding the following the following set of tuples to $F_{\mathcal{O}}$ each having weight 1: $\{(p_{\mathfrak{tt}}, \tau_l), (\tau_l, p_{\mathfrak{tt}}) \mid l \in L\}$.
    - For the case wherein $\mathfrak{tt} \in \mathcal{TR}^{\leq 2}_{\mathtt{tr}}$ we need to plugin the plus-gadgets for each $a \in A$ apart from the stop transition. Consider $P^{-1}(a) \subseteq L$, for each $a \in A$. For every $l \in P^{-1}(a)$ such that there is a transition $(l, a, A, b, l') \in tr$ we add a plus-gadget to $N_{\mathcal{O}}$ as follows: we add one extra transition $t^{(l,a,A,b,l')}_{\mathfrak{tt}}$ in $T_{\mathcal{O}}$ as shown in the Figure 1 and add the following tuples in the flow relation $F_{\mathcal{O}}$: $\{(l, t^{(l,a,A,b,l')}_{\mathfrak{tt}}), (p_{\mathfrak{tt}}, t^{(l,a,A,b,l')}_{\mathfrak{tt}}), (t^{(l,a,A,b,l')}_{\mathfrak{tt}}, p_{\mathfrak{tt}}), (t^{(l,a,A,b,l')}_{\mathfrak{tt}}, l')\}$ with each having weight 1.
- $\mathcal{V}_{\mathcal{O}}$, the *interpretation function*, is the same as V.

Figure 1 presents a representative joint action in our encoding scheme. Let $Runs(N_{\mathcal{O}})$ be the set of all runs starting from initial marking $M^0_{\mathcal{O}}$ for an encoded Petri net $N_{\mathcal{O}}$.

We define two maps, mf_1 and mf_2, one from from global states of $\mathcal{O}$ to the markings of $N_{\mathcal{O}}$ and the other from markings of $N_{\mathcal{O}}$ to the global states of $\mathcal{O}$. These definitions will be useful in the proof of correctness of the encoding scheme defined above.

Definition 6. *$mf_1 : G \longrightarrow \mathbb{M}_{\mathcal{O}}$ such that for any $g \in G$, $M = mf_1(g) \in \mathbb{M}_{\mathcal{O}}$ satisfying the following condition: for each $l \in L$, $M(l) =$ number of times l occurs in g and $M(g.E) = 1$.*

Definition 7. *$mf_2 : \mathbb{M}_{\mathcal{O}} \longrightarrow 2^G$ such that for any $M \in \mathbb{M}_{\mathcal{O}}$ and for every $g \in mf_2(M)$ the following condition holds: for each $l \in L$, $M(l) =$ number of times l occurs in g and $M(g.E) = 1$.*

Now, we assert the following claims:

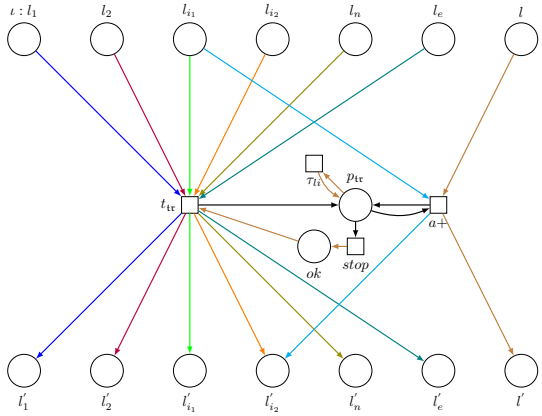

Fig. 1. Partial encoding of a joint action

Claim 1. *There is a map* $\mathfrak{h}_1 : Runs(\mathcal{O}) \longrightarrow Runs(N_\mathcal{O})$ *such that for every* $g \in G$ *if there is a run* $\rho \in Runs(\mathcal{O})$ *from* g_0 *to* g *then there is a run* $\varrho \in \mathfrak{h}_1(\rho)$ *from* $M^0_\mathcal{O}$ *to* $M = mf_1(g)$.

Proof: Let $\rho = g^0 g^1 g^2 \dots g^i \dots g^{\mathfrak{n}}$ be the run of $\mathcal{O}$, where $g^0 = g_0$ and $g^{\mathfrak{n}} = g$. For every $0 \leq i \leq \mathfrak{n}$, we map g^i to a sequence of markings in $N_\mathcal{O}$, $\varrho^i \in \mathbb{M}^+_\mathcal{O}$, as shown in the Figure 2. The concatenation of all such ϱ^i's gives us a legal run $\varrho^0 \cdot \varrho^1 \cdot \varrho^2 \cdots \varrho^i \cdots \varrho^{\mathfrak{n}} = \varrho \in Runs(N_\mathcal{O})$.

We construct ϱ inductively

- $n = 0$: $\varrho^0 = M^0_\mathcal{O}$ where $M_\mathcal{O}(\iota_E) = 1$.
- $n > 0$: Assume that ϱ^i is well defined for all $i < n$. Suppose $M^*_\mathcal{O}$ is the last configuration in the sequence ϱ^{n-1}. We need to do the following, for some $\mathfrak{m}_n$ that we compute later: define $\varrho^n = M^1_\mathcal{O} M^2_\mathcal{O} \dots M^{\mathfrak{m}_n}_\mathcal{O}$ such that $M^*_\mathcal{O} \longrightarrow M^1_\mathcal{O}$ and for all $j \in \{1, 2, \dots, \mathfrak{m}_n - 1\}$, $M^j_\mathcal{O} \longrightarrow M^{j+1}_\mathcal{O}$.
  As mentioned above, we define ϱ^n as a sequence of configurations $M^1_\mathcal{O} M^2_\mathcal{O} \dots M^{\mathfrak{m}_n}_\mathcal{O}$. We consider the three cases depending on the type of the transition (g^{n-1}, g^n) and define $M^j_\mathcal{O}$ for all $j \in \{1, 2, \dots, \mathfrak{m}_n\}$
  - (g^{n-1}, g^n) is an **agents joining transition**: In this case $\mathfrak{m}_n = 1$. $M^1_\mathcal{O}$ is obtained from $M^*_\mathcal{O}$ by incrementing the count for ι in $M^*_\mathcal{O}$ by 1: $M^1_\mathcal{O} = M^*_\mathcal{O}[\iota \mapsto M^*_\mathcal{O}(\iota)+1]$. Also, the label for the transition $M^*_\mathcal{O} \longrightarrow M^1_\mathcal{O}$ is t_{aj}.
  - (g^{n-1}, g^n) is an **agents leaving transition**: In this case $\mathfrak{m}_n = 1$. $M^1_\mathcal{O}$ is obtained from $M^*_\mathcal{O}$ by decrementing the count for $leave$ in $M^*_\mathcal{O}$ by 1: $M^1_\mathcal{O} = M^*_\mathcal{O}[leave \mapsto M^*_\mathcal{O}(leave) - 1]$. Also, the label for the transition $M^*_\mathcal{O} \longrightarrow M^1_\mathcal{O}$ is t_{al}.
  - (g^{n-1}, g^n) is a **transition with joint action** $\langle a^n, b \rangle$: Let $g^{n-1}, g^n \in G_z$. That is, there are z agents in the system at the time of joint action. Let $(g^{n-1}, \langle a^n, b \rangle, g^n) = \langle (l^{n-1}_1, a^n_1, l^n_1), \dots, (l^{n-1}_z, a^n_z, l^n_1), (l^{n-1}_e, b, l^n_e) \rangle$
    - after removing the id's that don't change across the joint action. Assume $Act = \{a_1, \dots, a_\kappa\} \cup \{\tau\}$. Suppose κ_0 actions in Act occur 0 times in a^n, κ_1 actions in Act occur 1 time and κ_2 actions occur ≥ 2 times in a^n. Assume further all τ actions figure at the right end of the sequence a^n and there are κ_τ of those.
    Without loss of generality assume that first $\kappa_2 + \kappa_1$ actions in a^n sequence are distinct. Further assume next κ_2 actions again are distinct. Now, consider the tuple $\langle (l^{n-1}_1, a^n_1, l^n_1), \dots, (l^{n-1}_{z'}, a^n_{z'}, l^n_1), (l^{n-1}_e, b, l^n_e) \rangle$, where $z' = 2\kappa_2 + \kappa_1$. This tuples constitutes a some $\mathfrak{tr}$ in $N_\mathcal{O}$. Now, consider the tuple $g^n_\dagger = \langle (l^n_1, id^n_1), \dots (l^n_{z'}, id^n_{z'}) \rangle$ and define $M^1_\mathcal{O} = mf_1(g^n_\dagger)$. Clearly, rest of the tuple (after removing $\mathfrak{tr}$ from $(g^{n-1}, \langle a^n, b \rangle, g^n)$) consists of zero or more a_+ transitions, say z'' of them, followed by κ_τ τ transitions. For each $2 \leq i \leq z'' + 1$, we read off $(l^{n-1}_{z'+i-1}, a_{z'+i-1}, l^n_{z'+i-1})$ from (g^{n-1}, a^n, g^n) and compute $M^i_\mathcal{O}$ from $M^{i-1}_\mathcal{O}$. Also, we define the corresponding transition label to be $t_{\mathfrak{tr}}^{(l^{n-1}_{z'+i-1}, a_{z'+i-1}, A, b, l^n_{z'+i-1})}$.
    Now, for all $1 \leq i \leq \kappa_\tau$, we can read off $(l^{n-1}_{z'+z''+i}, \tau, l^n_{z'+z''+i})$ from (g^{n-1}, a^n, g^n). We define $M^{1+z''+i}_\mathcal{O}$ for all $1 \leq i \leq \kappa_\tau$ to be same as $M^{1+z''}_\mathcal{O}$, which is already computed and define the transition label as $\tau^n_{1+z''+i}$ for all such i. Thus, we are done.

The way ϱ is constructed, as defined above, makes sure that it is a run in $N_\mathcal{O}$.

□

Claim 2. *There is a map* $\mathfrak{h}_2 : Runs(N_\mathcal{O}) \longrightarrow 2^{Runs(\mathcal{O})}$ *such that for every* $M \in \mathbb{M}_\mathcal{O}$ *if there is a run* $\varrho \in Runs(N_\mathcal{O})$ *from* $M^0_\mathcal{O}$ *to* M *then there is a run* $\rho \in \mathfrak{h}_2(\varrho)$ *from* g_0 *to some* $g \in mf_2(M)$.

Proof: Let $\varrho = M^0 \xrightarrow{t_0} M^1 \xrightarrow{t_1} M^2 \xrightarrow{t_2} \cdots \xrightarrow{t_{i-1}} M^i \xrightarrow{t_i} \cdots \xrightarrow{t_{\mathfrak{m}-1}} M^{\mathfrak{m}} = M$ be a run of $N_\mathcal{O}$ where $M^0_\mathcal{O} = M^0$ is the initial marking of $N_\mathcal{O}$. We can split the run ϱ into sequences $\varrho^i \in \mathbb{M}^+_\mathcal{O}$ and map them to g^i as shown in the Figure 3. Algorithm 1 gives us the unique split $\varrho^0, \varrho^1, \dots, \varrho^i, \dots \varrho^{\mathfrak{n}}$, where $\mathfrak{n} \leq \mathfrak{m}$. The split is primarily based on the transitions $\mathfrak{t}_j$'s in ϱ.

$\mathfrak{t}_j$ may be one of the following kinds

- t_{aj}; the corresponding ϱ^i will be a singleton.
- t_{al}; the corresponding ϱ^i will be a singleton.
- $t_{\mathfrak{tr}}$; the corresponding ϱ^i starts at this $\mathfrak{tr}_i$ and includes all the transitions (& markings) till we encounter a *stop*.
- $t^{\mathfrak{tr}}_{a+}$; this transition is part of ϱ^i of $t_{\mathfrak{tr}}$ preceding it.
- *stop*; this transition works as a delimiter for a joint action.

We construct $\rho = g^0 \cdot g^1 \cdots g^i \cdots g^{\mathfrak{n}}$ from the split $\varrho^0, \varrho^1, \dots, \varrho^i, \dots \varrho^{\mathfrak{n}}$ inductively as follows

- $n = 0$: $g^0 = \langle \iota_E \rangle$.
- $n > 0$: Assume that g^i is well defined for all $i < n$. For all $0 \leq i < n-1$, $(g^i, g^{i+1}) \in R$. We need to do the following: define g^n such that $(g^{n-1}, g^n) \in R$. Let us assume that there are z agents in g^{n-1}. Let $g^{n-1} = \langle (l^{n-1}_1, id^{n-1}_1), (l^{n-1}_2, id^{n-1}_2), \dots, (l^{n-1}_z, id^{n-1}_z), l^{n-1}_E \rangle$.

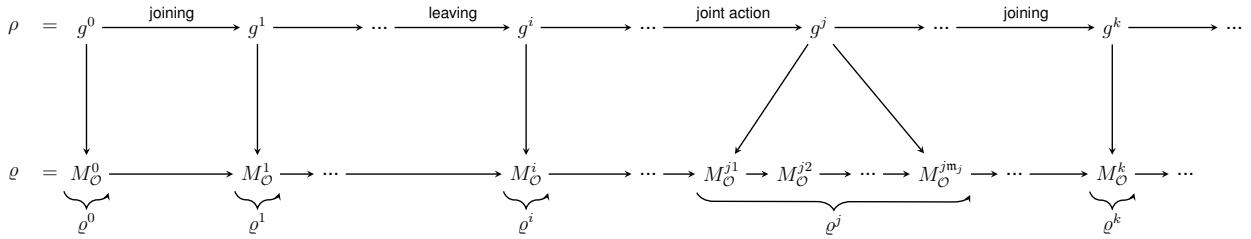

Fig. 2. Pictorial representation of construction of ϱ from ρ

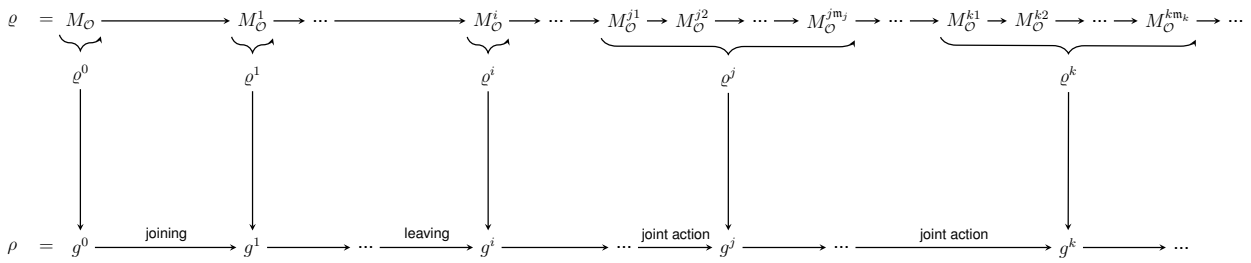

Fig. 3. Pictorial representation of construction of ρ from ϱ

Algorithm 1 Splitting the Run ϱ

$\varrho^0 \leftarrow M_{\mathcal{O}}^0$
$j \leftarrow 0$
for $i = 1$ **to** $\mathtt{m}$ **do**
  if $\mathfrak{t} = t_{\mathfrak{tr}}$ **then**
    $t_{\varrho^j} \leftarrow t_{\mathfrak{tr}}$
    $j \leftarrow j + 1$
    while $\mathfrak{t} = t_{a+}^{\mathfrak{tr}} \mid \tau_l \mid stop$ **do**
      $\varrho^j \leftarrow \varrho^j \cdot M^i$
      $i \leftarrow i + 1$
    end while
  else if $\mathfrak{t} = t_{aj}$ **then**
    $j \leftarrow j + 1$
    $\varrho^j \leftarrow M^i$
  else
    $j \leftarrow j + 1$
    $\varrho^j \leftarrow M^i$
  end if
  $i \leftarrow i - 1$
end for

Let M_p be the last marking in the sequence ϱ^{n-1} and M_q be first marking in the sequence ϱ^n. Let $\mathfrak{t}$ be the transition from $T_{\mathcal{O}}$ labeling $M_p \longrightarrow M_q$. We consider three cases, depending on the type of $\mathfrak{t}$

- $\mathfrak{t} = t_{aj}$: We construct g^n from g^{n-1} as follows. Let $\mathbb{D}$ be the set of IDs in g^{n-1}. Let id be the smallest integer in $\mathbb{N} - \mathbb{D}$. Then, $g^n = \langle (l_1^{n-1}, id_1^{n-1}), (l_2^{n-1}, id_2^{n-1}), \ldots, (l_z^{n-1}, id_z^{n-1}), (\iota, id), l_E^{n-1} \rangle$.
- $\mathfrak{t} = t_{al}$: There must be an agent in g^{n-1} in $leave$ state. Otherwise, g^{n-1} is not properly constructed. Suppose the index of this agent in $leave$ state in g^{n-1} is j. Then, we can construct g^n from g^{n-1} by removing the jth pair (l_j^{n-1}, id_j^{n-1}) from the tuple. That is, $g^n = \langle (l_1^{n-1}, id_1^{n-1}), \ldots, (l_{j-1}^{n-1}, id_{j-1}^{n-1}), (l_{j+1}^{n-1}, id_{j+1}^{n-1}), \ldots, (l_z^{n-1}, id_z^{n-1}), l_E^{n-1} \rangle$.
- $\mathfrak{t} = t_{\mathfrak{tr}}$: We consider two cases
  - $\mathfrak{t}$ is followed by $stop$: From $\mathfrak{tr}$, we collect the $(cur\_locn, action, nxt\_locn)$ tuples and combine them as follows: $\langle (l_1, \mathfrak{a}_1, l_1'), \ldots, (l_{\mathfrak{n}}, \mathfrak{a}_{\mathfrak{n}}, l_{\mathfrak{n}}'), (l_e, b, l_e') \rangle$. We observe that there is a permutation of this tuple wherein the pre-locations match the locations in g^{n-1}. From this permuted tuple, we read off g^n and a^n and we are done. Note that id's in g^{n-1} are to be carried over to g^n.
  - Otherwise: Suppose the sequence of transitions following $t_{\mathfrak{tr}}$ and before $stop$ are as follows: $t_1^n \cdot t_2^n \cdots t_\kappa^n$. Observe that for all $1 \leq i \leq \kappa$, t_i^n may be one of the two types
    - τ_l for some $l \in L$, which corresponds to $(l, \tau, l) \in tr$;
    - $t_{a^+}^{(l,a,A,b,l')}$ for some $a \in A$ and $(l, a, A, b, l') \in tr$.

    Now, from $\mathfrak{tr}$ and the t_i^n's we collect the $(cur\_locn, action, nxt\_locn)$ tuples and combine them as follows: $\langle (l_1, \mathfrak{a}_1, l_1'), \ldots, (l_{\mathfrak{n}}, \mathfrak{a}_{\mathfrak{n}}, l_{\mathfrak{n}}'), (l_1^{n-1}, a_1^n, l_1^n), (l_2^{n-1}, a_2^n, l_2^n), \cdots, (l_\kappa^{n-1}, a_\kappa^n, l_\kappa^n), (l_e, b, l_e') \rangle$. We observe that there is a permutation of this tuple wherein the pre-locations match the locations in g^{n-1}. From this permuted tuple, we read off g^n and a^n and we are done. Note, again, that id's in g^{n-1} are to be carried over to g^n.

It is easy to see that ρ as defined above, is a run in $\mathcal{O}$. □
The two claims proved above ensure that our encoding scheme is correct. Now we proceed to give a safety checking algorithm for regular OMAS.

Theorem 1. *Safety checking in regular OMAS is decidable.*

Proof: As a proof to the theorem, we reduce the problem of safety checking in regular OMAS to the *liveness* (*L1-live*) problem in Petri net.

A transition t in a Petri net (N, M_0) is said to be L1-live if t occurs at least once in some firing sequence in $\mathcal{L}(M_0)$.

Safety checking problem in regular OMAS can be defined as follows: Given a regular OMAS $\mathcal{O}$, we first define the notion of a global state g being *safe*. Let $g = \langle l_1, l_2, \ldots l_e \rangle$ be a global state in $\mathcal{O}$, g is safe if $\forall l \in g \setminus l_e$, $V(l) = \{safe\}$. If there is a location $l \in g \setminus l_e$ such that $V(l) = \{\}$ then g is labeled *unsafe*. A regular OMAS $\mathcal{O}$ with initial global state g_0 is said to be *safe* if every global state g reachable from g_0 is *safe*. Alternatively, no global state labeled *unsafe* is reachable.

Now the reduction of safety checking to *liveness* is as follows: Recall that a joint action in $\mathcal{O}$ corresponds to a transition $\mathfrak{t}$ in $N_{\mathcal{O}}$ of the form $\langle (l_1, \mathfrak{a}_1, A_1, b, l_1'), (l_2, \mathfrak{a}_2, A_2, b, l_2'), \ldots, (l_e, b, l_e') \rangle$ followed by plus-gadgets and/or τ transitions. Therefore, we first compute the set of unsafe transitions T_{unsafe} as given in Algorithm 2. Now for every $\mathfrak{t} \in T_{unsafe}$, we check whether $\mathfrak{t}$ is *L1-live* in $(N_{\mathcal{O}}, M^0_{\mathcal{O}})$ or not. If any of the $\mathfrak{t}$'s is *L1-live*, then we conclude that $\mathcal{O}$ is *unsafe*. Otherwise, if all the $\mathfrak{t}$'s are not *L1-live* then $\mathcal{O}$ is *safe*.

□

Algorithm 2 T_{unsafe} from $T_{\mathcal{O}}$

```
T_unsafe ← ∅
for each t ∈ T_O do
    for each tp ∈ t do
        if V(tp[5]) = {} then
            T_unsafe ← T_unsafe ∪ t
            break
        end if
    end for
end for
return T_unsafe
```

Now, we will see how to check the *L1-liveness* of a transition $\mathfrak{t}$. Let $\mathrm{m}_{\mathfrak{t}}$ be the minimum marking needed to enable the transition $\mathfrak{t}$. If $\mathrm{m}_{\mathfrak{t}}$ is reachable/coverable from the initial marking, meaning that $\mathfrak{t}$ occurs at least once in some firing sequence in $\mathcal{L}(M^0_{\mathcal{O}})$, then $\mathfrak{t}$ is *L1-live*. The detailed procedure is provided in Algorithm 3.

IV. Implementation and Experimental Evaluation

To validate the encoding of regular OMAS into Petri nets (described in Section III), we developed a tool named *Rutwiya SafeCheck*. The tool automates the translation of agent and environment specifications into Petri nets, and subsequently performs safety verification using *its-reach* from the ITS-tools suite *its-tool*. This section outlines the architecture of the implementation and presents the experimental results.

Algorithm 3 Safety Checking from T_{unsafe} and $M^0_{\mathcal{O}}$

```
M_{T_unsafe} ← ∅
if T_unsafe = ∅ then
    return Safe
end if
for each t ∈ T_unsafe do
    m_t ← minimum marking needed to enable the transition t
    M_{T_unsafe} ← M_{T_unsafe} ∪ m_t
end for
for each m_t ∈ M_{T_unsafe} do
    if m_t is reachable from M^0_O then
        return Unsafe
    end if
end for
return Safe
```

A. Tool Architecture

Rutwiya SafeCheck is implemented in Python. The major components are:

1) **Input Parser:** Reads a structured specification of the regular OMAS, including agent states, actions, protocols, and the designated leave state, as well as environment behavior.
2) **Transition Generator:** Constructs local transitions for agents and the environment, and systematically aggregates them into global transitions as per the encoding described in Section III.
3) **Petri Net Encoder:** Maps global states and transitions into the Guarded Action Language (GAL) representation, which is then translated into a Petri net model compatible with ITS-tools.
4) **Verification Backend:** Invokes *its-reach* to perform reachability analysis and determine whether any unsafe state is reachable from the initial configuration.

B. Experimental Setup

All experiments were conducted on an Apple MacBook Air (M1, 2020) with an 8-core CPU, 7-core GPU, 8GB unified memory, and 256GB SSD. The software environment included Python 3.10 and ITS-tools version 3.0. The benchmarks consist of synthetic regular OMAS instances with varying numbers of agents, environment states, and transitions.

C. Benchmark Scenarios

To evaluate usability and effectiveness, we considered test cases [1] where

- number of agent states and actions varied between 4–6,
- the environment contained 1–3 states, with up to 32 transitions and
- safety properties were specified as forbidden states.

D. Results and Discussion

Table I summarizes the encoding time, verification time, and safety outcomes for the test cases.

[1]Our tool and test cases are available at https://safetychecking.netlify.app/

TABLE I
EXPERIMENTAL RESULTS OF RUTWIYA SAFECHECK

Test Case	#States (Agents, Env.)	#Trans.	Enc. Time	Verif. Time	Result
1	(4, 1)	1	< 0.01s	< 0.2s	Unsafe
2	(4, 1)	4	< 0.02s	< 0.37s	Unsafe
3	(4, 1)	2	< 0.01s	< 1s	Unsafe
4	(4, 3)	12	< 0.8s	< 20s	Unsafe
5	(4, 3)	32	< 1s	< 2s	Safe
6	(4, 3)	2	< 0.05s	< 0.8s	Unsafe

The results indicate that encoding time remains negligible across all test cases, while verification time grows moderately with the number of environment states and transitions. For example, Test Case 5, which includes 32 transitions, was verified within 2 seconds, demonstrating the efficiency of the ITS-tools backend. Our codes are available via https://github.com/ipranavprashant/ReachabilityAndSafetyCheckingTool.

V. RELATED WORK

Verification of open multi-agent systems (OMAS) presents significant challenges due to their dynamic nature and inherent complexity. Kouvaros and Lomuscio [10] addressed these challenges by proving the undecidability of the general verification problem for OMAS through a reduction from uniform termination problem in lossy counter systems [11]. To overcome this fundamental limitation, they proposed two specialized subclasses—collective open interpreted systems (COIS) and interleaved open interpreted systems (IOIS)—which admit decidable verification problems. Among these, COIS is particularly relevant to our present work. The COIS subclass imposes constraints on agent templates by ensuring that the local transitions of each agent depend solely on its current state and the collective action executed by the entire system. Kouvaros and Lomuscio adapted decision procedures from their previous work [6], successfully establishing decidability primarily for homogeneous swarms, though the authors suggest possible extensions to heterogeneous multi-agent scenarios.

Our proposed subclass, regular OMAS, differs fundamentally from COIS by enforcing orthogonal constraints. Specifically, regular OMAS restrict agent departures to designated leave states rather than imposing conditions on transition functions. This crucial structural distinction enables a straightforward encoding into multi-counter systems and, consequently, Petri nets—an encoding strategy not directly applicable to COIS due to their differing constraints.

The utilization of counters in system abstraction has a long-standing history. Lubachevsky [7] introduced the concept of employing process counters to simplify the analysis of concurrent systems as early as 1984. Emerson and Trefler [8] further refined this approach by introducing the concept of *generic representatives*, enabling symmetry reduction in symbolic model checking.

Counter abstraction emerged explicitly as a concept in parameterized verification through the influential work of Pnueli et al. [14]. Their method abstracted unbounded parameterized systems into finite-state representations, facilitating practical verification. Kouvaros and Lomuscio [15] leveraged similar counter abstraction techniques specifically for robotic swarm verification. Moreover, counter abstraction has found success beyond theoretical studies, significantly influencing practical software model checking. Basler et al. [16] effectively utilized counter abstraction methods to reduce redundancy in concurrent program verification by abstracting replicated thread behavior.

Automated approaches for parameterized model checking have also been extensively explored. Emerson and Namjoshi [12] introduced fully automated techniques for synchronous systems, proving decidability for systems composed of homogeneous user processes and a centralized control mechanism when properties are expressed in indexed propositional temporal logic. German and Sistla [13] and Emerson and Namjoshi [17] similarly demonstrated decidability in special cases of parameterized verification, showcasing the feasibility of automatic solutions.

Recent related work includes Delporte-Gallet et al. [18], who employed counters within a non-negotiating distributed computing framework. In their model, processes increment private counters stored in shared memory to collaboratively solve multidimensional approximate agreement tasks, despite weak asynchronous conditions and crash failures. Although their application domain differs from ours, the fundamental use of counters to track system state progression resonates closely with the techniques employed in our approach.

Lastly, Alcantara et al. [19] have addressed coordination challenges within asynchronous robotic environments under the Look-Compute-Move model. Their work tackles autonomous robot coordination without global synchronization, highlighting the necessity of effective local decision-making mechanisms—a concept closely aligned with our interest in verifying dynamically evolving multi-agent scenarios.

(Parts of the contents of this section were paraphrased by [23]).

VI. CONCLUSION AND FUTURE WORK

In this paper, we work with a subclass of open MAS [10] called regular OMAS where an agent leaves the system only from its leaving state [5]. We had already shown reachability in such systems to be decidable by an encoding into multi-counter systems [5]. As multi-counter systems are not suitable for tool building, we defined a fresh encoding of regular OMAS into Petri nets and check safety in such systems via a reduction to L1-liveness in nets. We implemented a basic version of this encoding as a safety verifying tool called Rutwiya SafeCheck which can be accesses online https://github.com/ipranavprashant/ReachabilityAndSafetyCheckingTool.

As an immediate followup to the current work, we would like to implement an advanced version of Rutwiya SafeCheck that uses the full encoding described in this paper. In the present implementation we have used its-reach [4] as a back-end, instead we could replace it by KReach [20] or some other coverability checker to make safety verification more efficient. Furthermore, following the work of [16], we may explore the possibility of enhancing our safety checker by interleaving the translation process (to Petri nets) and safety checking process.

On the theory side, we would like to identify a subclass of regular OMAS that is amenable to full model checking (safety and liveness checking). In that case, we need to isolate a subclass of Petri nets, for which (full) model checking problem is decidable, and show its equivalence to the identified subclass of regular OMAS. Also, we would like to compute the exact complexity of safety checking problem of regular OMAS, which we expect to be better than EXPSPACE – the current complexity.

ACKNOWLEDGMENTS

The authors thank Prof. Prakash Saivasan (IMSc, Chennai) for his invaluable help in generalizing the Petri net encoding of regular OMAS. Further, we acknowledgement the help of ChatGPT in paraphrasing parts of the text of *Introduction* and *Related Work* sections.

REFERENCES

[1] S. R. Kosaraju, "Decidability of reachability in vector addition systems (preliminary version)," in *Proc. 14th Annu. ACM Symp. Theory Comput. (STOC)*, San Francisco, CA, USA, pp. 267–281, May 1982.
[2] E. W. Mayr, "An algorithm for the general Petri net reachability problem," *SIAM J. Comput.*, vol. 13, no. 3, pp. 441–460, 1984.
[3] T. Murata, "Petri nets: Properties, analysis and applications," *Proc. IEEE*, vol. 77, no. 4, pp. 541–580, Apr. 1989.
[4] Y. Thierry-Mieg, "Symbolic model-checking using ITS-tools," in *Proc. Tools Algorithms Constr. Anal. Syst. (TACAS)*, Berlin, Germany, pp. 231–237, 2015.
[5] A. T. Shabana, A. George, and S. Sheerazuddin, "Counter abstraction for regular open teams," *Inf. Process. Lett.*, vol. 188, p. 106533, 2025.
[6] P. Kouvaros and A. Lomuscio, "Verifying emergent properties of swarms," in *Proc. 24th Int. Joint Conf. Artif. Intell. (IJCAI)*, Buenos Aires, Argentina, pp. 1083–1089, Jul. 2015.
[7] B. D. Lubachevsky, "An approach to automating the verification of compact parallel coordination programs I," *Acta Informatica*, vol. 21, pp. 125–169, 1984.
[8] E. A. Emerson and R. J. Trefler, "From asymmetry to full symmetry: New techniques for symmetry reduction in model checking," in *Correct Hardware Design Verification Methods (CHARME)*, Bad Herrenalb, Germany, pp. 142–156, Sept. 1999.
[9] R. Fagin, J. Y. Halpern, Y. Moses, and M. Y. Vardi, *Reasoning About Knowledge*. Cambridge, MA, USA: MIT Press, 1995.
[10] P. Kouvaros, A. Lomuscio, E. Pirovano, and H. Punchihewa, "Formal verification of open multi-agent systems," in *Proc. 18th Int. Conf. Autonomous Agents MultiAgent Syst. (AAMAS)*, Montreal, QC, Canada, pp. 179–187, May 2019.
[11] P. Schnoebelen, "Lossy counter machines decidability cheat sheet," in *Proc. Int. Workshop Reachability Problems (RP)*, Brno, Czech Republic, pp. 51–75, Aug. 2010.
[12] E. A. Emerson and K. S. Namjoshi, "Automatic verification of parameterized synchronous systems (extended abstract)," in *Proc. 8th Int. Conf. Computer Aided Verification (CAV)*, New Brunswick, NJ, USA, pp. 87–98, Jul.–Aug. 1996.
[13] S. M. German and A. P. Sistla, "Reasoning about systems with many processes," *J. ACM*, vol. 39, no. 3, pp. 675–735, 1992.
[14] A. Pnueli, J. Xu, and L. D. Zuck, "Liveness with (0, 1, ∞)-counter abstraction," in *Proc. 14th Int. Conf. Computer Aided Verification (CAV)*, Copenhagen, Denmark, pp. 107–122, Jul. 2002.
[15] P. Kouvaros and A. Lomuscio, "A counter abstraction technique for the verification of robot swarms," in *Proc. 29th AAAI Conf. Artif. Intell. (AAAI)*, Austin, TX, USA, pp. 2081–2088, Jan. 2015.
[16] G. Basler, M. Mazzucchi, T. Wahl, and D. Kroening, "Symbolic counter abstraction for concurrent software," in *Proc. 21st Int. Conf. Computer Aided Verification (CAV)*, Grenoble, France, pp. 64–78, Jun.–Jul. 2009.
[17] E. A. Emerson and K. S. Namjoshi, "Reasoning about rings," in *Proc. 22nd ACM SIGPLAN-SIGACT Symp. Principles Program. Lang. (POPL)*, San Francisco, CA, USA, pp. 85–94, Jan. 1995.
[18] C. Delporte-Gallet, H. Fauconnier, P. Fraigniaud, S. Rajsbaum, and C. Travers, "Non-negotiating distributed computing," in *Proc. 31st Int. Colloq. Structural Information and Communication Complexity (SIROCCO)*, Vietri sul Mare, Italy, pp. 208–225, May 2024.
[19] M. Alcantara, A. Castañeda, D. Flores-Peñaloza, and S. Rajsbaum, "The topology of look-compute-move robot wait-free algorithms with hard termination," *Distrib. Comput.*, vol. 32, no. 3, pp. 235–255, 2019.
[20] A. Dixon and R. Lazic, "KReach: A tool for reachability in Petri nets," in *Proc. 26th Int. Conf. Tools Algorithms Construction Anal. Syst. (TACAS)*, Dublin, Ireland, pp. 405–412, Apr. 2020.
[21] M. Luckcuck, M. Farrell, L. A. Dennis, C. Dixon, and M. Fisher, "Formal specification and verification of autonomous robotic systems: A survey," *ACM Comput. Surv.*, vol. 52, no. 5, pp. 100:1–100:41, 2019.
[22] https://lip6.github.io/ITSTools-web/reach.html#obtaining-its-reach
[23] https://chatgpt.com/